Programmeringsuppgift 3

Getting started

All submissions should be implemented in Go unless the problem specifies something different. You can download Go at https://go.dev/dl/. You can also use various package managers to install it, e.g., HomeBrew on macOS.

Cite any relevant sources or approaches that you used in order to understand and solve the problems.

Objective

Part A) Create a client for a HTTP-server.

Part B) Create a set of Go modules that you tie together using the Go build system via *go.mod*-files and a correct folder/package structure.

Part A

Download srv.go and test.py from Moodle. Your task is to create a client for the provided server. The server is a key-value store where you can store keys (put), get the value stored for a key (key), and delete a key (delete). The operations return reasonable values: 200 for success, 201 for a created key, and 404 for errors.

Your client should support functions for the three operations and a main program that shows that the client works. Use the Python test program as inspiration.

You are allowed to change the server, e.g., the port it runs on, and add functionality. Note that the client does not support concurrent access to the keys (put). This should be trivial to fix with, e.g., an RWMutex or sync.Map.

Part B

Part B.1

Follow the tutorial on creating a Go Module from the go.dev homepage:

https://go.dev/doc/tutorial/create-module

Question:

- In the tutorial, besides the *already mentioned hitch* in the chapter about *greeting multiple people*, why did they create a new function? What is the reason that we can not simply overload the Hellos() function?

Part B.2

Objective

Create a Go program that uses functionality in a couple of standalone modules. Imagine being part of a team of developers where each team is responsible for one module.

Tasks

Main program

- Create a main program (e.g. *lnu_configurator*) and create separate modules that can calculate properties of basic electric component types, such as *resistors* and *capacitors*. See the modules section below for more details.
- In your program ask the user to a) select one of the component types and b) ask the user to input the known properties. Your program should then c) call the relevant function in the corresponding module to calculate the unknown. Ask the user to configure a total of 3 components. You decide freely what types using which module functionality.
- Finally, add functionality in your program that uses the *os*-module to create a textfile. It should contain a very basic summary of the 3 configured electric components (as chosen by the user and calculated by each module). The file should be written when the user decides to quit the application. Show the contents of the file when the user restarts the application - as a reminder of the previous session.

Modules

- Create a few modules with code for calculating basic properties of electronic components. Implement support for 2 to 3 common component types.
- In each module, implement a couple of functions to be used for calculating the *unknown property* of an electric component, given the known properties. Let me explain:

  - For example, in a *resistor* module you could have a function called *calcEffect()* to which you supply two arguments: the subjected *voltage* [V] and *amperage* [A] on a resistor in an electric circuit. The function should return the *effect* [W] using the equation P = U * I. Here, the voltage and amperage are the known properties that the user feds into the program.
  - Another function in your module might instead use the relationship U = I * R. Here, two typical functions could be *calcAmperage()* or *calcVoltage()*.

- For a *capacitor* module you could look up the formula for capacitance and create suitable functions.
- Demonstrate the use of the functions in the main program.

Reflection

- Can you come up with a few useful unit tests for your modules before you start coding? Try to imagine being part of team of developers that develop each module separately. Implement and demonstrate their use if you like.

Optional exercises

- Cross-platform build
  - How do you build your program for another platform?
    * What OS/platforms can you build for with your version of Go? How can you find out?
  - If you have access to a different platform, then make a build targeting that platform. Verify that your program works as intendend.
    * You can use/install one of the tools/environments mentioned in the Systems API lecture. If your machine can handle it, then WSL should be fairly straightforward.

Tips

List the Go environment variables used using:

```
go env
```

Hand-in

Submit your solutions as a single zip file via Moodle.

This is a group assignment that can be done in groups of one or two students. Your submission should contain well-structured and organized Go code for the problems with a README.txt (or .md) file describing how to compile and run the Go programs and a short PDF-report describing your experiment, findings and any struggles.